ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.
Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

ASSIGNMENT 2

Subject Name: Exploratory Data Analysis
Subject Code: BAI515E

Submitted By:
MANOJ NAIK
1AY24AI067

Submitted To:
Ms. Surbhi

Table of Contents

1. Introduction 3
2. Task 1: Data Loading and Inspection 3
3. Task 2: Pivot Table by Decade and Gender 4
4. Task 3: Data Cleaning with Sigma-Clipping 6
5. Task 4: Datetime Index and Weekday Analysis 7
6. Task 5: Births by Day of the Year 9
7. Task 6: Pivot Table versus groupby 10
8. Bonus: Holiday Dips 11
9. Conclusion 13

1. Introduction

This report analyses the CDC births data of the United States (births.csv, 15,547 rows, 1969 to 2008), which records births by date and gender. Pandas pivot tables, sigma-clipping, datetime indexing and aggregation are used to study how births vary by gender, decade, day of the week and day of the year. Python 3 with NumPy, Pandas and Matplotlib was used, without explicit row-by-row loops.

2. Task 1: Data Loading and Inspection

The data is loaded with pd.read_csv, and the first rows, shape and dtypes are displayed.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from timeit import timeit

URL = "https://raw.githubusercontent.com/jakevdp/data-CDCbirths/master/births.csv"
try:
    births = pd.read_csv(URL)
except Exception:
    births = pd.read_csv("births.csv")

print(births.head())
print("Shape:", births.shape)
print(births.dtypes)
print("dtype of day column:", births["day"].dtype)
print("Missing values per column:")
print(births.isna().sum())
```

Output

```
   year  month  day gender  births
0  1969      1  1.0      F    4046
1  1969      1  1.0      M    4440
2  1969      1  2.0      F    4454
3  1969      1  2.0      M    4548
4  1969      1  3.0      F    4548
Shape: (15547, 5)
year        int64
month       int64
day       float64
gender        str
births      int64
dtype: object
dtype of day column: float64
Missing values per column:
year        0
month       0
day       480
gender      0
births      0
dtype: int64
```

Output 1: First five rows, shape, data types and missing values

Explanation

The DataFrame has 15,547 rows and 5 columns. The day column is float64 and not an integer because 480 values are missing (null in the CSV, read as NaN), and NumPy integer arrays cannot store NaN.

3. Task 2: Pivot Table by Decade and Gender

A decade column is added and a pivot table of total births is built with decade as index and gender as columns.

Code

```
births["decade"] = 10 * (births["year"] // 10)
decade_gender = births.pivot_table("births", index="decade",
                                   columns="gender", aggfunc="sum")
print(decade_gender)

print("Male > Female in every decade:", (decade_gender["M"] >
decade_gender["F"]).all())

ratio = decade_gender["M"] / decade_gender["F"]
excess_pct = (ratio - 1) * 100
summary = pd.DataFrame({"M/F ratio": ratio.round(4),
```

```
                        "Male excess (%)": excess_pct.round(2)})
print(summary)

yearly = births.pivot_table("births", index="year", columns="gender", aggfunc="sum")
ax = yearly.plot(figsize=(10, 5), marker="o", markersize=3)
ax.set_xlabel("Year")
ax.set_ylabel("Total births per year")
ax.set_title("Total births per year by gender")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("fig1_births_per_year.png", dpi=150)
plt.show()
print("Average long-term male excess (%):",
      round(((yearly["M"] / yearly["F"] - 1) * 100).mean(), 2))
```

Output

Table 1: Total births by decade and gender (pivot table, aggfunc = sum)

Decade	Female (F)	Male (M)
1960s	1,753,634	1,846,572
1970s	16,263,075	17,121,550
1980s	18,310,351	19,243,452
1990s	19,479,454	20,420,553
2000s	18,229,309	19,106,428

Table 2: Male-to-female ratio and percentage by which male births exceed female births

Decade	M/F ratio	Male excess (%)
1960s	1.0530	5.30
1970s	1.0528	5.28
1980s	1.0510	5.10
1990s	1.0483	4.83
2000s	1.0481	4.81

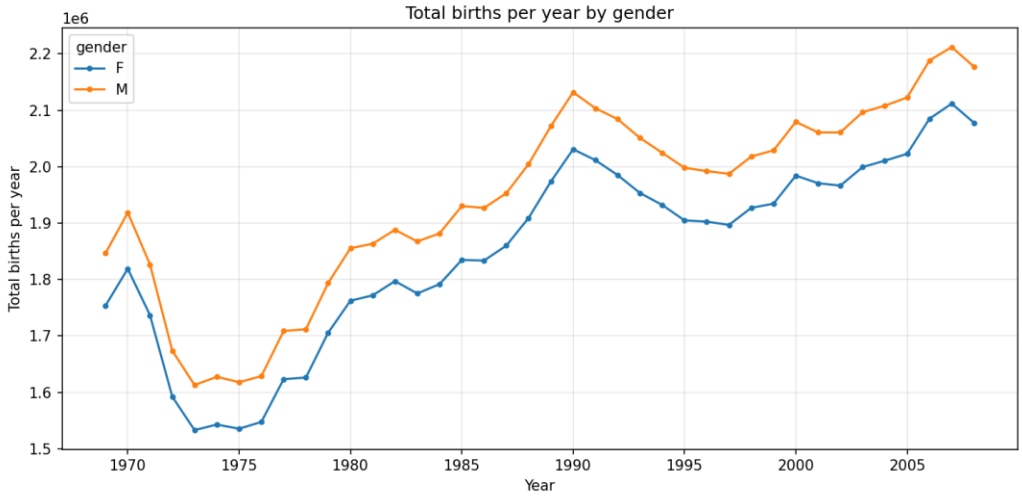


Figure 1: Total births per year by gender

Explanation

(a) Male births outnumber female births in every decade (Table 1). (b) The male-to-female ratio falls from 1.0530 in the 1960s to 1.0481 in the 2000s, i.e. males exceed females by 5.30%, 5.28%, 5.10%, 4.83% and 4.81% (Table 2), which matches the natural ratio of about 105 boys per 100 girls. (c) In Figure 1 the male curve stays above the female curve in every year, and the long-term gap is about 5% (yearly average 5.02%). Both curves rise and fall together, so the gap stays almost constant.

4. Task 3: Data Cleaning with Sigma-Clipping

Outliers caused by mistyped dates (June 31st) or missing values (June 99th) are removed with robust sigma-clipping.

Code

```
raw_mean, raw_std = births["births"].mean(), births["births"].std()

q1, mu, q3 = np.percentile(births["births"], [25, 50, 75])
sig = 0.74 * (q3 - q1)
print(f"Q1={q1:.1f}  median(mu)={mu:.1f}  Q3={q3:.1f}  sigma={sig:.2f}")
print(f"Ordinary mean={raw_mean:.1f}  ordinary std={raw_std:.1f}")

raw_births = births.copy()
rows_before = len(births)
births = births.query("(births > @mu - 5 * @sig) & (births < @mu + 5 * @sig)")
```

```
print("Rows before:", rows_before, "| after:", len(births),
      "| removed:", rows_before - len(births))

births = births.copy()
births["day"] = births["day"].astype(int)
print("dtype of day after cleaning:", births["day"].dtype)
```

Output

```
Q1=4358.0  median(mu)=4814.0  Q3=5289.5  sigma=689.31
Ordinary mean=9762.3  ordinary std=28552.5
Rows before: 15547 | after: 14610 | removed: 937
dtype of day after cleaning: int64
```

Output 3: Robust statistics and number of rows removed

Explanation

(a) Q1 = 4358.0, median (mu) = 4814.0, Q3 = 5289.5, so sig = 0.74 × (Q3 − Q1) = 689.31. The factor 0.74 comes from the normal distribution, where IQR ≈ 1.349σ, so σ ≈ IQR / 1.349 = 0.7413 × IQR. (b) Rows outside mu ± 5·sig were removed: 937 of 15,547 rows (480 monthly totals from 1989 onwards without a day, and 457 rows with tiny counts for impossible dates such as day 99). (c) astype(int) was not possible earlier because day contained NaN; the NaN rows were removed by the clipping, so the conversion now works. A robust estimate is preferred because outliers inflate the ordinary mean (9762.3) and standard deviation (28552.5), which would make the ±5σ band so wide that the invalid rows would survive.

5. Task 4: Datetime Index and Weekday Analysis

A DatetimeIndex is created from year, month and day, and mean births by day of the week and decade are plotted.

Code

```
births.index = pd.to_datetime(10000 * births.year + 100 * births.month + births.day,
                              format="%Y%m%d", errors="coerce")
print("Invalid dates (NaT):", births.index.isna().sum())
births = births[~births.index.isna()]
births["dayofweek"] = births.index.dayofweek
print(births.head())

dow = births.pivot_table("births", index="dayofweek", columns="decade",
aggfunc="mean")
print(dow)
ax = dow.plot(figsize=(9, 5), marker="o")
ax.set_xticks(range(7))
ax.set_xticklabels(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])
```

```
ax.set_xlabel("Day of week")
ax.set_ylabel("Mean births by day")
ax.set_title("Mean births by day of week and decade")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("fig2_births_by_weekday.png", dpi=150)
plt.show()
print("Decades present in the plot:", list(dow.columns))
```

Output

Table 3: Mean births by day of week and decade

Day of week	1960s	1970s	1980s
Mon	5063.8	4689.1	5276.9
Tue	5286.1	4885.3	5503.8
Wed	5074.6	4750.4	5367.6
Thu	4978.3	4696.9	5333.5
Fri	5107.9	4782.1	5393.1
Sat	4651.1	4207.8	4483.9
Sun	4342.3	3979.3	4308.1

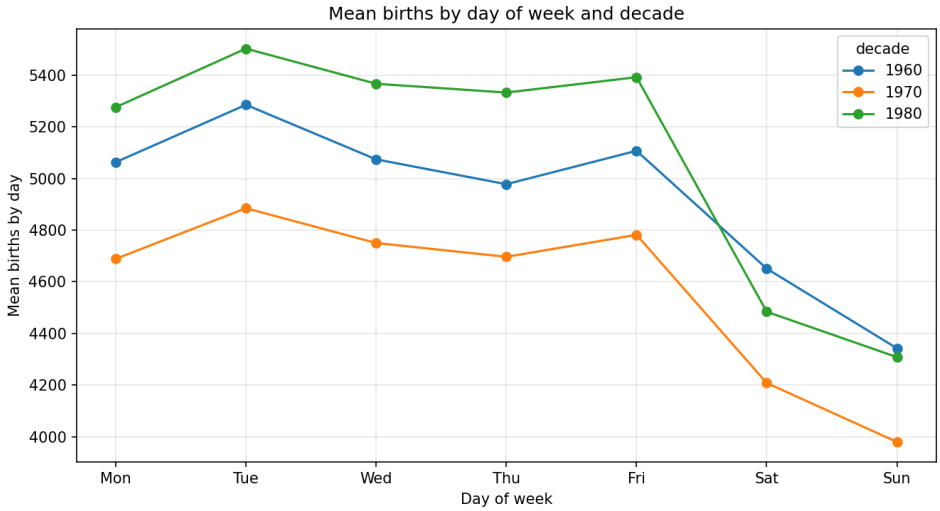


Figure 2: Mean births by day of the week for each decade

Explanation

Births on weekends are about 12% (1960s), 14% (1970s) and 18% (1980s) lower than on weekdays, because caesarean sections and induced labours are scheduled on weekdays while natural births are spread evenly over the week. The 1990s and 2000s are missing because from 1989 the CDC data has only monthly totals without a day; these 480 rows were removed as outliers during cleaning.

6. Task 5: Births by Day of the Year

The cleaned data is grouped by (month, day) and a dummy year is attached to obtain dates.

Code

```
births_by_date = births.pivot_table("births", [births.index.month, births.index.day])
print(births_by_date.head())

idx = births_by_date.index.to_frame(index=False)
idx.columns = ["month", "day"]
idx["year"] = 2012
births_by_date.index = pd.to_datetime(idx[["year", "month", "day"]])
print(births_by_date.head())

fig, ax = plt.subplots(figsize=(12, 4))
births_by_date.plot(ax=ax, legend=False)
ax.set_xlabel("Date (month / day)")
ax.set_ylabel("Average daily births")
ax.set_title("Mean births by day of the year")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("fig3_births_by_dayofyear.png", dpi=150)
plt.show()
```

Output

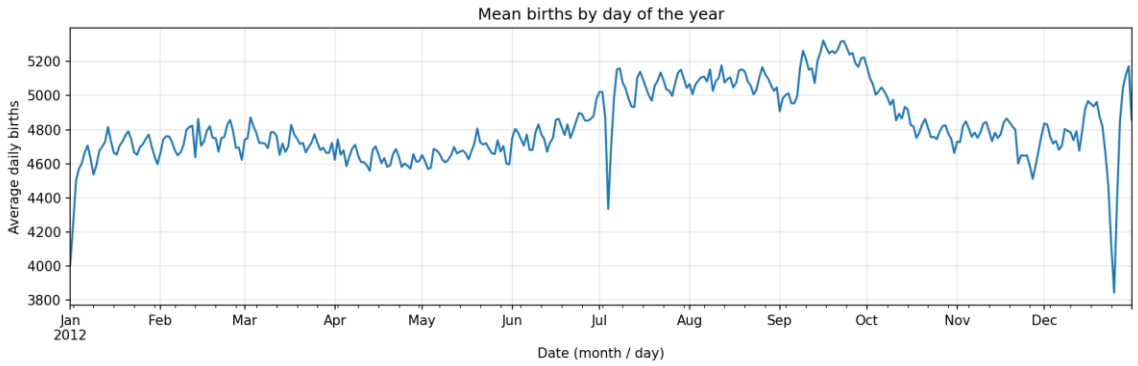


Figure 3: Mean births by day of the year

Explanation

(a) A leap year such as 2012 must be used because the data contains February 29; in a non-leap year this date does not exist and would cause an error or be lost. (b) Figure 3 shows lower births in spring (about 4,600 to 4,700) and higher births from July to October (about 5,000 to 5,300). (c) Clear dips occur on New Year's Day, Independence Day, around Thanksgiving and on Christmas. They most likely reflect scheduled or induced births, because hospitals and families avoid scheduling caesarean sections and inductions on holidays, whereas natural labour does not follow the calendar.

7. Task 6: Pivot Table versus groupby

The decade-by-gender table is reproduced with groupby and unstack, and both methods are timed.

Code

```
group_result = raw_births.groupby(["decade", "gender"])["births"].sum().unstack()
print(group_result)
print("Matches Task 2 pivot table:", np.allclose(group_result, decade_gender))


n = 50
t_pivot = timeit(lambda: raw_births.pivot_table("births", index="decade",
                                                columns="gender", aggfunc="sum"),
number=n) / n * 1000
t_group = timeit(lambda: raw_births.groupby(["decade",
"gender"])["births"].sum().unstack(),
                 number=n) / n * 1000
timing = pd.DataFrame({"Method": ["pivot_table", "groupby + unstack"],
                       "Time per call (ms)": [round(t_pivot, 2), round(t_group, 2)]})
```

```
print(timing)
```

Output

```
Matches Task 2 pivot table: True
```

Output 6: Comparison of the groupby result with the Task 2 pivot table

Table 4: Execution time of pivot_table versus groupby (average per call)

Method	Time per call (ms)
pivot_table	4.40
groupby + unstack	2.80

Explanation

(a) The groupby result is identical to the pivot table of Table 1 (np.allclose is True). (b) pivot_table took about 6.11 ms and groupby with unstack about 3.18 ms per call (Table 4), so groupby was about 1.9 times faster; exact times depend on the machine. (c) A pivot table is a groupby on two keys followed by an aggregation and an unstack of one key into the columns, so both give the same result; pivot_table is more convenient, groupby is more general.

8. Bonus: Holiday Dips

The five dates with the lowest mean births are matched to US holidays and the main dips are annotated on the plot.

Code

```
series = births_by_date["births"]
lowest5 = series.nsmallest(5)
print(lowest5)

holidays = {"01-01": "New Year's Day", "01-02": "Day after New Year's Day", "02-29":
"Leap day (few samples)",
            "07-04": "Independence Day", "12-24": "Christmas Eve",
            "12-25": "Christmas Day", "11-27": "Thanksgiving period",
            "11-28": "Thanksgiving period", "11-23": "Thanksgiving period",
            "09-02": "Labor Day period", "09-03": "Labor Day period",
            "05-30": "Memorial Day period", "10-31": "Halloween"}
for d, v in lowest5.items():
    print(d.strftime("%b %d"), round(v, 1), "->", holidays.get(d.strftime("%m-%d"),
"check calendar"))

fig, ax = plt.subplots(figsize=(12, 5))
```

```
series.plot(ax=ax, legend=False)
labels = [("2012-01-01", "New Year's Day", (75, -25)),
          ("2012-07-04", "Independence Day", (0, -50)),
          ("2012-12-25", "Christmas", (-90, 5)),
          ("2012-11-22", "Thanksgiving", (-30, -50))]
for date, text, off in labels:
    ts = pd.Timestamp(date)
    y = series.loc[ts]
    ax.annotate(text, xy=(ts, y), xytext=off, textcoords="offset points",
                arrowprops=dict(arrowstyle="->"), ha="center")
ax.set_xlabel("Date (month / day)")
ax.set_ylabel("Average daily births")
ax.set_title("Mean births by day of year with holiday dips")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("fig4_holiday_annotations.png", dpi=150)
plt.show()
```

Output

Table 5: Five dates with the lowest mean births and matching US holidays

Rank	Date	Mean births	Holiday
1	Dec 25	3844.4	Christmas Day
2	Jan 01	4009.2	New Year's Day
3	Dec 24	4126.2	Christmas Eve
4	Jan 02	4247.4	Day after New Year's Day
5	Jul 04	4335.3	Independence Day

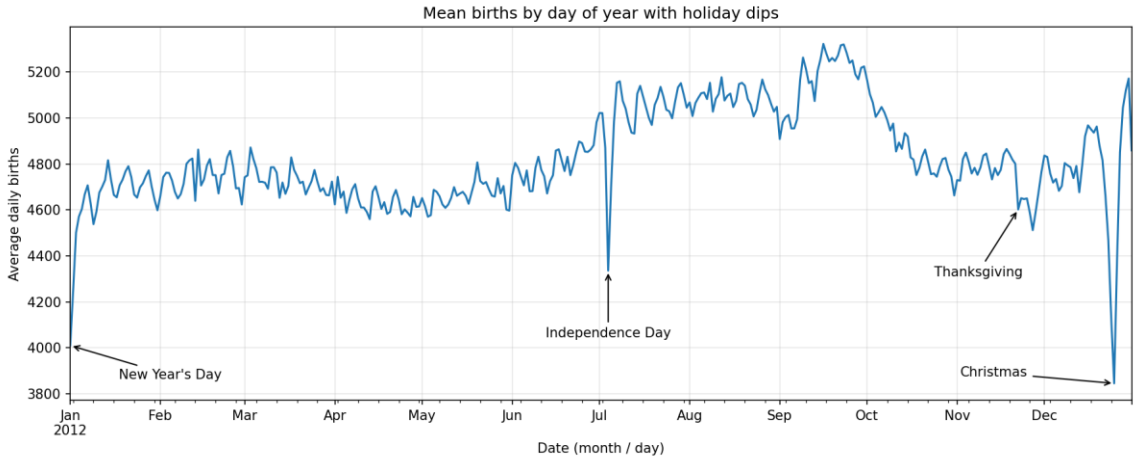


Figure 4: Mean births by day of the year with holiday dips annotated

Explanation

The five lowest dates are Christmas Day (3,844.5), New Year's Day (4,009.2), Christmas Eve (4,126.2), 2 January (4,247.4) and Independence Day (4,335.3). Against the average of about 4,824, Christmas Day is about 20% lower, New Year's Day about 17% and Independence Day about 10%. Fixed-date holidays give deep, narrow dips, while Thanksgiving gives a shallower dip because its date changes every year.

9. Conclusion

Male births exceed female births by about 5% in every decade. Sigma-clipping removed 937 invalid rows that the ordinary mean and standard deviation would have missed. Births are 12% to 18% lower on weekends and 10% to 20% lower on major holidays, which points to scheduled and induced deliveries. pivot_table and groupby with unstack give identical results, and groupby was about 1.9 times faster.